

函数级代码生成的预算分档编排：从单步到级联的统一实现研究

薛沁晨

摘要

在 API 计费与上下文长度约束下，函数级代码生成的部署效果取决于测试时计算如何组织，而非单次生成成本本身。本文将问题表述为可复现的预算分档编排协议：在单一实现与统一统计口径下，把从单步到级联的行为固定为四个可对照档位，使不同预算形态可在同一执行器与同一评测口径下比较。

主实验在 HumanEval 与 MBPP 两条函数级基准上联合报告通过率、调用次数与总 token，并给出经验收益率 ρ 刻画各档边际与累计成本-收益关系。结果表明：在更难、更杂的任务分布上，分档编排带来更宽的通过率提升空间；在易饱和和分布上，增益更多体现为残差段收束与成本重分配。机制上，静态结构难度分更适合作为预算分配先验而非运行失败分类器；multiset 随机路由对照用于分离 hard 覆盖率与结构对齐的贡献；反馈回注形态对照表明，压缩回注在精度几乎不变的前提下可显著降低 token 开销。

部署上，中预算侧重结构化主链与失败回注修补，高预算侧重候选扩展并在可选 refine 上做精度与 token 的折中。

关键词：函数级代码生成；测试时计算；预算分档；失败修补；级联编排

Abstract

Under API pricing and context-length constraints, the deployment effectiveness of function-level code generation depends on how test-time computation is organized, rather than on a single generation attempt alone. This paper formulates the problem as a reproducible **budget-tiered orchestration protocol**: under one implementation and one unified accounting interface, behaviors from single-step generation to cascaded workflows are fixed into four comparable tiers, so that different budget forms can be compared under the same executor and evaluation protocol.

The main experiments jointly report pass rate, number of calls, and total tokens on the HumanEval and MBPP function-level benchmarks, and introduce an empirical gain rate ρ to characterize both marginal and cumulative cost-benefit relationships across tiers. Results show that tiered orchestration provides broader room for pass-rate improvement on harder and more heterogeneous task distributions; on easier and more saturated distributions, the gains are more reflected in residual-set convergence and cost reallocation. Mechanistically, static structural difficulty scores are more suitable as **budget-allocation priors** than as runtime failure classifiers; the multiset random-routing control separates the contribution of hard-slot coverage from structural alignment; and the feedback-injection comparison shows that compressed feedback can substantially reduce token cost while keeping accuracy nearly unchanged.

For deployment, the medium-budget setting emphasizes a structured main chain and failure-feedback repair, whereas the high-budget setting emphasizes candidate expansion and trades off accuracy against token cost through optional refine steps.

Keywords: function-level code generation; test-time computation; budget tiering; failure repair; cascade orchestration

1 引言

函数级代码生成在部署侧首先受 API 预算约束：调用次数、上下文长度与 token 费用共同决定了测试时计算的上限 [1, 2]。在这一约束下，系统要回答的核心问题不是“模型能否偶尔写出可运行程序”，而是如何把有限预算分配到更可能产生边际收益的环节 [3, 4, 5, 6, 7]。对强模型而言，大量样本已可被单次生成覆盖；编排的价值在于把剩余预算投向失败形态更混杂、更需要修补或候选扩展的样本段。

本文用**预算分档编排**来刻画这一问题：在**同一实现中**，用统一入口把策略形态固定为四个可解释的预算档位，形成从**单步到级联**的递进链条——轻量单步、单步失败修补、难度分流下的重链与失败救援、以及高预算级联扩展。这样写的目的，是把比较从「模式 A 对模式 B」的松散对照，收紧为「**同一实现下逐级解锁不同计算投向**」的阶梯式对照，并自然引出后文对调用与 token 的解释需求。

失败样本并非同质：有的失败更适合用紧凑执行反馈做局部修补，有的失败则需要更广的候选探索与筛选；也可能出现「继续追加阶段但边际收益不足」的情形。由此，「多阶段」并不天然等于「更好」[3, 7]。预算分档编排的意义，正在于让读者能够追问：**每一档多花的 token 究竟买到了哪一类能力**，而不是把复杂度本身误认为贡献。

本文正是在这一背景下展开：不把难度路由、失败修补与级联扩展写成彼此割裂的技巧清单，而把它们写成同一阶梯上的**预算形态递进**——中档强调「该不该进重链」与「失败信息如何回注」，高档强调「候选集如何扩展与组织」。本文希望说明的是，代码生成系统的提升不只来自「算得更多」，更来自**算得更对症**。

本文的贡献按“**一条主线 + 三层机制证据**”组织。主线贡献是：在单一实现与统一统计口径下给出可复现的**预算分档协议**，并在双基准上系统呈现**通过率-调用次数-token**前沿与部署可用的 ρ 指标。机制证据包括：其一，用结构难度分与阈值 τ 形成可解释的 easy/hard 预算分配先验，并说明静态子分**并不等价于运行失败预测器**；其二，给出 **multiset 随机路由整卷对照**，用于分离 hard 覆盖率与结构对齐的贡献；其三，给出失败反馈回注形态的多种子对照，刻画压缩回注相对完整 traceback 的精度与 token 差异。

论证结构。全文按“协议设计 → 机制验证 → 成本前沿”展开：第 3 节给出四档编排协议与设计动机；第 4 节先报告双基准端到端结果，再用控制消融验证关键机制，最后系统呈现经验收益率 ρ 与部署可用的成本收益前沿；第 5 节给出讨论与结论。

2 相关研究

围绕代码生成的测试时增强，现有研究可归为三条与本文直接相关的线索。第一条是**评测与口径**：HumanEval 与 MBPP 构成函数级代码生成的主评测基线 [1, 2]，APPS、CodeXGLUE、DS-1000、EvalPlus、LiveCodeBench 等扩展了任务分布与测试覆盖 [8, 19, 11, 9, 10]。第二条是**执行反馈驱动修补**：Self-Debug、Reflexion、CRITIC、Self-Refine、AgentCoder、CodeT 等工作显示，反馈是否被压缩为可执行约束，直接影响修补收益上限 [4, 5, 6, 13, 12, 20]。第三条是**测试时计算分配**：AlphaCode 及后续 test-time scaling 路线证明，候选扩展与筛选可以提升通过率，但代价是显著的 token 与调用增长 [3, 7]。

上述工作分别回答“在哪评测”“如何修补”“如何扩展计算”，但对工程部署仍有一个共同空白：当系统同时启用分流、修补与级联时，如何在**同一实现中**保证可比性，避免由模板差异、默认开关与采样口径造成的伪增益。本文据此将问题重心从“新增单模块”转为“**统一协议下的预算分配**”，即在统一执行器、统一统计与统一入口下，比较不同预算形态的真实边际收益。

因此，本文在相关工作中的定位是**可部署协议研究**而非单点技巧改进：主文关注成本-收益前沿，机制层关注可复现实证，最终目标是把预算决策映射为可量化的 token 代价与可执行部署建议。

3 方法

本文在**统一实现**（同一执行器、同一通过判定与统计口径、同一编排入口）下实现**预算分档编排**：总体形态见图 1，按题执行的端到端流水线及四档差异见第 3.1 节。其后先给出问题边界与成本记账（第 3.2 节），再分节展开难度分流、失败救援与级联等模块，用以解释各档内部的计算投向。

本文的方法设计围绕一个基本判断展开：代码生成中的额外测试时计算，只有在能够显著缩小后

续搜索空间时才具有实际价值 [4, 5, 6, 7]。因此，系统并未把预算简单地理解为「增加若干次调用」，而是将其分配到三个彼此衔接的环节中：首先判断哪些样本值得进入重路径，其次在第一次失败后提取能够约束后续生成的局部错误信息，最后在更高预算下将计算扩展到候选探索与筛选层面。如图 1 所示，本文方法的核心不在于无差别地增加阶段，而在于让不同预算档位对应不同的信息利用方式，使各档具有清晰的功能分工，而不是演化为若干边界含混的工程开关。

3.1 总体流程与预算分档

对每道任务 x_i ，系统在实现上遵循同一外层闭环：读入题面及（对模型可见的）测试接口信息 $\rightarrow$ 语言模型生成候选程序 $y_i \rightarrow$ 在沙箱中执行单测并得到通过/失败 $\rightarrow$ 若失败，则仅在当前预算档位所允许的范围内追加后续调用（修补、重链、救援或级联），直至通过或达到该档策略的上限。通过率、平均调用次数与总 token 均在同一统计管道中汇总，从而保证四档结果可比。

分档由单一入口控制：实现中将离散档位编码与仓库内模式名一一对应，避免「模式名」与「预算档位」两套开关并存带来的比较歧义。映射见表 1；完整命令行见仓库复现说明。

各档在行为上可概括为：

- **单步**：一次生成即提交判定；该档不把失败样本继续送入本文后续多轮编排分支，用作轻量基线与残差起点。
- **单步 + 修补**：在单次生成失败后，以执行反馈为约束进行有限轮次的修补（轻路径；对应 `single_step_repair` 形态）。
- **分流 + 重链与救援**：先依据结构难度分数 s_i 与阈值 τ 划分 easy/hard；easy 侧保持相对轻量，hard 侧进入更重主链（可含 spec/cot 等开关），并在增强修复阶段按需触发候选救援与 refine 救援；失败信息经压缩后进入修补/替换等闭环（见第 3.4 节）。
- **级联**：在更高预算下扩展多候选生成、执行筛选与 refine；主实验采用级联默认基线，将预算集中于主级联链路（见第 3.5 节）。

同一外层闭环在四档下复用相同的执行与判定逻辑；各档差异主要体现在失败后的追加调用是否发生、以何种提示形态发生、以及是否引入多候选

级联。下文第 3.2 节给出问题边界与成本记账，其后按模块说明分流、救援与级联。

3.2 问题定义与成本记账原则

需要强调的是，上述编排逻辑与具体基准数据集**正交**：同一套执行器、同一套分档入口与同一统计口径同时作用于 HumanEval 与 MBPP；差异只体现在任务分布与难度天花板上，而这正是双基准并列评测的意义——观察同一编排是否在“易饱和”与“更难、更杂”的两类代码生成场景下给出一致的趋势与可解释的成本结构。

设数据集为 $\mathcal{D} = \{x_i\}_{i=1}^N$ ，其中 x_i 为一道代码生成任务。系统在给定预算档位 b 下生成候选 y_i ，经执行器得到 $z_i = E(y_i) \in \{0, 1\}$ 。本文关心的不是写出抽象的约束优化闭式解，而是在可测的记账口径下比较各档：对每题记录是否通过、调用次数与总 token；主文以总 token 作为 API 成本主代理（与第 4.4 节 ρ 一致），调用次数作为辅助指标。单次生成提供最低档基线，并决定后续各档所面对的残差失败分布——后续阶段并非替代单步，而是对残差样本做有预算上限的追加计算。

本文不在正文引入带数值权重 α, β 的联合成本泛函；若需同时按次计费与按 token 计费，可将实验中的 ρ 定义改为「在固定 α/β 下的边际收益」，作为部署扩展而非本文主表默认。

3.3 难度感知路由

难度感知路由的作用，是在进入重路径之前完成一次粗粒度预算筛分。系统依据题面与测试可见结构特征，对样本进行 easy/hard 划分，使明显简单样本尽量停留在轻路径，而将更需要额外推理或修补的样本送入重路径。这里追求的不是完美预测器，而是避免把同等预算平均花在所有样本上。

记样本 x_i 的结构难度分数为 s_i 。实现上由 `_difficulty_score_v` 给出可分解启发式：五类子分数分别刻画题面复杂度、签名/接口复杂度、测试用例复杂度、边界条件密度与题面歧义程度；另叠加有界体量项 e_i （由题面/测试长度、行数、断言行数、candidate 调用次数等经对数与线性缩放后相加并上限截断，避免总分分布漂移）。离线统计表与主程序使用同一分解，便于附录核对。

$$s_i = p_i + \sigma_i + \tau_i + b_i + a_i + e_i, \quad (1)$$

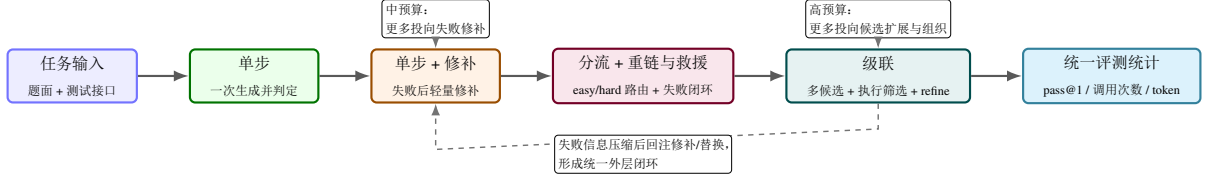


图 1: 预算分档编排的整体框架（统一实现）。系统以单步生成作为起点，经失败修补与难度分流进入重链，再在更高预算下扩展到候选探索、执行筛选与后续 refine，形成从单步到级联的递进。主实验在 HumanEval 与 MBPP 上复现同一套分档流程与统计口径，用以检验该链条是否随题型分布与难度天花板而漂移。

其中 $p_i, \sigma_i, \tau_i, b_i, a_i$ 分别对应题面、签名、测试、边界与歧义五类子项， e_i 为体量增量。实现上，五类子分项各自在代码中经上界裁剪后相加得到基础分，再叠加有界体量项 e_i ；各子项内部由若干启发式规则与固定正权重线性组合（例如题面长度分段、关键词命中按次加权并封顶、assert 行计数分段、candidate 形参与边界词表加权等），总分为确定性函数而非学习到的预测器。相对权重体现为：测试结构与边界信号在总分中通常占比较高，题面歧义与体量项起校准尺度作用；子分项与结构分档、运行标签的统计关系见附录表 12。本文将路由解释为预算分配规则：用 s_i 与 τ 控制 hard 覆盖率，从而在精度与 token 之间形成可调前沿（阈值消融见实验节）。

路由决策写为

$$r_i = \mathbb{I}(s_i \geq \tau_{\text{eff}}), \quad (2)$$

其中 τ_{eff} 与配置中的名义结构阈值一致（当前分流 + 重链与救援档默认取 1.6）；实现仅统计分数分布（最小/中位/最大）供分析，不对阈值做“分布地板抬升”。 $r_i = 1$ 表示样本进入 hard 主链。在 tiered 等模式下还可叠加 LLM 评审等信号，本文主表仍聚焦于上述结构驱动的分档端到端效果。

Multiset 随机路由对照（实现口径）。为分离「hard 槽位覆盖率」与「槽位与题面结构的对应关系」，仓库提供 `--difficulty_assignment=random_permutation`，并返回到下一轮生成中。先按式 (1) 与 τ_{eff} 得到本批任务上的 easy/hard 标签 multiset（各题 structural 判定），再在固定随机种子下对该标签序列做均匀随机置换，使每题被赋予 easy 或 hard 的个数与 structural 完全一致，但题一槽位对应被打乱；随后仍执行同一套分流 + 重链与救援协议。该对照不是「任意 token 预算下的均匀随机路由」，而是在固定 hard 覆盖率 multiset 下检验结构对齐是否带来额外收益，与 LR 离线证据回答的问题正交互补。

可复现性与随机性。随机种子固定任务顺序与

部分随机模块，但生成调用仍采用非零温度，同配置重复运行可能出现个别样本通过与否或总 token 的波动；表 3 报告的是三 seed 的均值与样本标准差。部分补充表为控制算力仍为单次 seed，正文在对应表下注明。

关于体量项与附录统计。 e_i 的作用是让总分与题面、测试规模保持弱对齐，便于路由分数在不同题型下保持可解释尺度。附录中的相关性与逻辑回归结果用于描述这一结构信号与运行标签的关系；主结论仍由同协议端到端结果给出，即通过率、调用次数与 token 的联合变化。

这一阈值设定使语义保持稳定：调高阈值使更多样本停留在 easy 链，调低则更多进入 hard 链；后文阈值消融亦建立在“直接阈值、直接分流”的实现上。

3.4 失败后救援链

难度感知只能决定哪些样本进入重路径，但并不能保证额外预算转化为有效收益。为此，本文在分流 + 重链与救援档的重路径中引入失败后救援链。其思路不是简单重试，而是将执行失败转化为后续生成可利用的局部约束 [4, 5, 6]。系统先执行候选程序，再从运行结果中提炼与修补直接相关的错误信息，并回注到下一轮生成中。

这一设计与普通重复采样的根本区别在于，后续调用不再面对原始问题，而是面对一个已经被错误信息裁剪过的更窄搜索空间。对代码生成任务而言，第一次失败通常已经暴露出相当具体的局部结构，例如参数个数不匹配、断言不满足、边界条件处理错误，或实现主体基本正确但中间步骤存在偏差。若这些信息能够被压缩为紧凑且具有约束性的反馈，则第二次生成更可能表现为局部修补，而不是从头重写。

基于这一考虑，本文将救援链组织为“生成、执

行、回写错误、再生成”的短闭环，并强调对错误信息的结构化整理，而不是直接复制全部运行日志。实现上，系统在 `repair` 与 `replace` 之间按错误类型分流：当失败形态呈现“局部可修复”特征时，如断言差异、边界条件遗漏、变量更新错误，优先走 `repair`；当失败形态表现为“整体方案偏离”时，如函数骨架与测试意图不匹配、核心思路错误或重复修补仍不收敛，转入 `replace`。该分流的目的，是把额外调用优先用于最可能产生增量的路径，避免把全局重写与局部修补混在同一提示中。

分流机制与输入。 `repair/replace` 标签由规则函数对执行器返回的 `check_correctness` 结果字符串做关键词判定得到（非额外模型调用）：例如未闭合字符串、`InvalidSyntax` 等归入 `replace`；缺参、参数个数不匹配、`NameError/IndentationError` 等可归入 `repair`；其余默认 `replace`。在 `direct` 与 `spec` 双路径都失败时，实现还比较两条路径错误标签是否一致，以决定后续修补轮数为 1 或 2（一致视为“易定位”，不一致视为“分歧、难定位”）。第三档进一步提供两级救援——候选救援与 `refine` 救援，可分别开关。其中 `hard_spec` 的角色是为 `hard` 路径提供结构化约束提示，即接口、边界与测试意图，使第一次重链生成先收窄解空间再进入后续救援；因此它属于主链约束模块而非后处理修饰模块。记第一次失败反馈为 $e_i^{(1)}$ ，压缩后的回注包为

$\phi(e_i^{(1)}) = \text{Concat}(\text{错误类型, 截断摘要, 规则片段, 定向修补提示})$

(3) 读者可将四项理解为：归类信号、可放入提示的短错误摘要、从历史失败归纳的修补规则、以及指向具体位置的修补提示（实现中由统一格式化函数打包）。因此第二轮生成上下文可表示为

$$\tilde{x}_i^{(2)} = \text{Concat}(x_i, \phi(e_i^{(1)})). \quad (4)$$

相应地，第二轮候选程序满足

$$y_i^{(2)} \sim p_\theta(\cdot | \tilde{x}_i^{(2)}). \quad (5)$$

这一写法强调的不是模型形式本身，而是搜索空间在执行反馈作用下发生了变化。原始 `traceback` 往往冗长且包含大量与修补无直接关系的细节，未经处理地回注模型容易重新放大搜索空间。该主张在表 7 的 E2 对照中获得独立检验：在仅替换回注形态、其余流程不变时，`compressed` 与 `raw_traceback` 的五 `seed` 均值通过率差异很小，而 `compressed` 的平均总 `token` 更低。因而，失败后救援的关键不在于

是否追加了一次调用，而在于是否将追加预算转化为更强的局部约束。

从整体方法上看，分流 + 重链与救援档之所以应当被视作一条独立路径，正是因为它改变了额外预算的作用位置。相比于仅在题目层面继续扩展推理，失败后救援把预算进一步下沉到了错误层面的局部修补之中。这种变化虽然在流程图上只体现为少量附加步骤，但对系统搜索行为的影响是实质性的。

3.5 高预算级联 workflow

级联模块 (`workflow_cascade`) 的设计目标不是继续加深单样本修补链，而是把新增计算用于候选空间覆盖。其架构可分为三层：候选扩展层（生成多候选并拉开解空间）、执行筛选层（在统一执行器下按测试结果过滤）、后处理层（仅对进入候选集的样本做有限 `refine`）。该设计把“生成-验证-再组织”写成显式流水线，避免把多阶段流程退化为无差别重试 [3, 7]。

实现上，级联主链按以下顺序执行：(1) `direct` 基线候选给出低成本起点；(2) `extra` 候选扩展在失败或低置信样本上追加候选集合；(3) 执行筛选按统一判定口径选择最优可行候选；(4) `refine/fuse` 仅在候选池已形成可行解时触发一次性微调。该顺序的目的，是保证主要预算先用于“扩展可行解覆盖率”，而非提前消耗在后处理上。

候选数量、可行解与 `refine`。可行解指在给定任务上于统一执行器下通过官方单元测试的候选实现。`extra` 阶段每题最多追加的生成次数由全局上限 `max_repair_rounds` 与路由表共同约束：在可学习路由的自适应档位绑定中，保守/平衡/进取档分别对应最多约 1/2/3 个额外生成，并对执行器实际执行的 `top-k` 做同档绑定（例如进取档最多执行前 3 个生成候选）；启发式路由下仍受 CLI 默认 `extra_exec_topk` 等下限约束，实际执行数为 $\min(\text{生成数}, \text{top-k})$ 。`refine_once` 仅在开启相应 CLI 开关且路由允许时触发，用于在已形成可行候选池后对最优候选做一次微调；保守档可在路由表中关闭 `refine` 以省 `token`。

与第 3.4 节的失败后救援链相比，级联模块的职责边界是清晰分开的：救援链解决“已知失败样本如何被局部约束修复”，级联模块解决“在不确定解空间中如何提高命中率”。因此，级联并不是救援链的机械叠加，而是把计算投向从局部修补切换到候

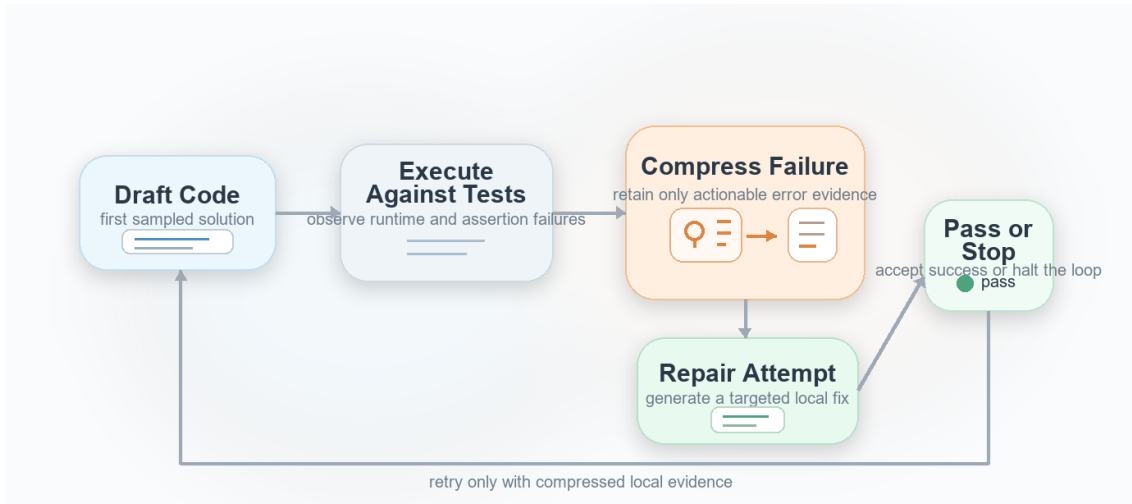


图 2: 失败驱动救援链的核心闭环。与简单重试不同，系统并不直接重复原始任务，而是先执行候选程序，再将错误信息压缩为更紧凑的局部约束，用于指导下一轮生成。

选覆盖与筛选组织；两者解决的是不同层级的问题。

实现上，级联模块遵循“先筛选、后精修”的触发顺序：只有候选在执行筛选后保留，才进入 refine；当候选扩展未带来有效可行解时，流程在该层停止而不继续放大后处理调用。结合实验节级联开关消融，extra（候选扩展）提供主增益，refine 相关开关主要负责精度-成本微调。方法上应将级联理解为“用候选覆盖换取上限”，而非对中等预算链路的逐层替代。

3.6 预算分档与模块映射

综合而言，预算分档编排将从单步到级联组织为四个可对照层级：单步提供轻量起点；单步 + 修补把计算投向可压缩的执行反馈；分流 + 重链与救援在难度分流下决定重链覆盖范围，并由失败后救援把追加调用转化为局部约束；级联则把计算主要投向候选扩展、执行筛选与 refine。各档差异对应的是预算投向与信息利用方式，而非孤立工程开关。

这一点也是本文实验比较成立的前提：读者在阅读表 3 时，应始终把每一行理解为同一实现下某一档位的端到端形态，从而把性能差异解释为分档链条与任务分布的相互作用，而不是孤立模块的偶然波动。

4 实验

4.1 实验设置与协议

主线实验在 HumanEval（164 题）与 MBPP sanitized（427 题）两条标准函数级代码生成基准上并行进行，用以检验预算分档编排之统一实现在从单步到级联路径上是否给出可复现、可解释的趋势：前者更贴近易饱和的短函数设定；后者提供更宽的改进空间与更可分辨的成本曲线。核心主结果模型统一为 claude-sonnet-4-6。预算划分为四档语义档位，并与表 1 中的实现模式一一对应（single_step、single_step_repair、difficulty_aware_plus、workflow_cascade）。保持「预算档位 = 策略形态」，避免模式组合带来的解释歧义[1,2]。第三档在双基准上共用 difficulty_aware_plus 实现；难度打分采用 v2 结构分。具体命令行写法见仓库复现说明。主表采用三 seed（42、78、132）均值 ± 样本标准差；生成阶段存在非零温度，同配置重复仍可能出现个位数样本波动，因此正文结论以区间统计与前沿形态共同解读。

消融与主线的分工。表 3 与表 4、表 5 覆盖双基准、多模型的端到端结论；表 8-10 的细粒度开关实验仅在 MBPP 上展开，用于解释分流 + 重链与救援档与级联档内部模块贡献与成本前沿。这样安排的首要原因是评测动态范围：在本文主模型设定下，HumanEval 通过率已接近饱和和区间，细粒度消融更常表现为“个位数样本差异”，对模块贡献排序与阈值前沿的统计稳定性不足；MBPP 则保留更宽的改进空间，更适合承载机制解剖。其次才是 API 成本

表 1: 语义档位与实现模式映射（正文口径）

语义档位	实现模式	主要计算投向
单步	<code>single_step</code>	单次生成与执行判定
单步 + 修补	<code>single_step_repair</code>	执行反馈约束下的轻量修补
分流 + 重链与救援	<code>difficulty_aware_plus</code>	难度分流、hard 主链与失败后救援
级联	<code>workflow_cascade</code>	多候选扩展、执行筛选与 refine

注：第三档在双基准上共用同一实现 `run_mbpp_difficulty_aware_plus`；命令行预算档位别名为 `difficulty_aware_repair`，仅用于复现，不改变协议定义。第一档日志别名 `base_direct` 与 `single_step` 同义。

与可重复性。主结果已在 HumanEval 上验证同一编排的分档趋势，MBPP 消融进一步回答“趋势背后的机制与预算花在哪里”。

除非特别说明，MBPP 结果采用 MBPP sanitized 的 427 题口径，HumanEval 结果采用 164 题口径；主结果表（表 3）中的各方法共享同一执行脚本与同一统计方式。凡涉及外部论文结果的横向对照，均单独标注并仅作背景映衬，不参与主结论的严格同条件比较。

评价指标除了通过率之外，还同时记录平均调用次数和总 token 数。本文特别强调后两者，是因为在真实 API 场景中，调用次数并不能完整代表成本。一次很长的失败分析提示，可能比两次短调用更昂贵；而多阶段系统真正难以控制的，往往恰恰是上下文累积带来的 token 膨胀。因此，凡是涉及“方法更划算”或“预算更有效”的判断，都必须结合 token 视角来理解。就评价思想而言，这种写法与近年来关于代码生成测试时扩展的讨论是一致的，即不只比较最终正确率，也较为获得这部分正确率所付出的额外计算 [3, 7]。

为避免“复杂流程天然占优”的偏差，本文将公平基线限定为同轮次或同预算可比形式：其一是固定采样次数的简单重试/pass@k 基线（即在相同候选规模下比较选择机制）[1]；其二是固定总 token 开销的 best-of-N 基线（在相近计算预算下比较流程收益）[7]。此外，主结果采用同模型、同数据切分、同执行器与同统计口径的设置；主表（表 3）为三 seed 区间统计，多模型分档表与多数 MBPP 消融表为单次 seed=42 的形态扫描，二者证据权重不同（见表 2）。

公平基线按「可直接复现 + 外部可对照」两层组织：外部方法优先选取 Self-Repair 相关且文献成熟的路线（如 Self-Debug、Reflexion、CodeT）用于量级参考；同协议主对比仅使用仓库内可复现实现，

避免跨论文协议不一致导致的假比较。

4.2 主结果

本节按「主结果 → 多模型对照 → 机制验证 → 成本前沿」组织：先给出双基准端到端趋势，再检验同一分档结构在不同基础模型上的形态稳定性，最后通过 MBPP 控制消融、双基准 multiset 随机路由对照与 ρ 指标给出可部署的预算前沿证据。

表 3 给出预算分档编排在 HumanEval 与 MBPP 上的端到端表现（主模型为 claude-sonnet-4-6）。表中策略列以档位语义名为准，括号内仅给出与实现模式相关的关键标识（如默认级联基线、 τ 等），便于读者对照第 3.1 节描述；完整复现命令见仓库说明。

HumanEval 上，claude-sonnet-4-6 在单步档已接近高位，三 seed 下分档增益主要体现为残差段收束：从单步到单步 + 修补即可获得主要增益，后续档位更多体现为上限逼近与成本重分配。该趋势说明在易饱和基准上应联合阅读通过率区间与 token 前沿，而不宜只看单次百分点差。

MBPP 上，同一套分档呈现更宽的动态区间：从单步到级联，通过率与 token 同步抬升的趋势更显著。这与题面更长、失败形态更混杂的分布相一致，也更适合作为观察「预算投向是否划算」的主场景。两相对照，可把本文贡献读作：统一实现下的分档编排并不总是表现为「任何基准都巨幅涨分」，而常表现为在更难、更杂的分布上提供可扩展收益，同时在易饱和的分布上仍保持合理的成本曲线与残差提升。

针对 MBPP，四档均已补齐三 seed 统计并回填主表：从单步到单步 + 修补的边际 ρ 最高，后续档位边际 ρ 递减但累计通过率继续抬升，呈现“先以中档建立高性价比增益、再以高档追求上限”的前

表 2: 实验协议约束 (fairness constraints)

约束项	本文约束
数据口径	MBPP sanitized 427 + HumanEval 164
模型与版本	主结果统一 <code>claude-sonnet-4-6</code> ; 多模型结果单独列出
随机性控制	主表 MBPP+HumanEval 四档三 seed 统计, 42、78、132; E2 反馈形态对照补充五 seed, 42、78、132、256、512; multiset 随机路由对照为 seed=42 单次 (structural 与 random_permute_matched 各一次)
执行与判定	统一执行器、超时、测试调用方式与通过判定口径
比较轴线	主结论以同协议内 预算分档编排 双基准主结果 (表 3, 三 seed) 为准; 多模型分档表为 seed=42 单次形态扫描; 细粒度消融 (除 E2 五 seed 外) 多为 seed=42 单次
轮次公平	报告固定候选规模 (<code>pass@k/simple retry</code>) 对比
预算公平	报告固定总 token (<code>best-of-N</code>) 对比
成本指标	同时报告通过率、平均调用次数、总 token, 避免单指标偏置

表 3: 预算分档编排主结果 (统一实现; 模型: `claude-sonnet-4-6`; MBPP+HumanEval)

数据集	策略	通过率 (%)	平均调用次数	总 token	边际 ρ
MBPP	单步 (<code>single_step</code>)	82.51 ± 0.48	1.000 ± 0.000	$113,922 \pm 5,621$	—
MBPP	单步 + 修补 (<code>single_step_repair</code>)	86.65 ± 0.24	1.177 ± 0.001	$141,207 \pm 4,393$	1.52
MBPP	分流 + 重链与救援 (<code>difficulty_aware_plus</code>)	88.52 ± 0.24	1.483 ± 0.009	$218,327 \pm 2,138$	0.24
MBPP	级联 (<code>workflow_cascade</code>)	91.72 ± 0.36	1.639 ± 0.021	$321,177 \pm 11,854$	0.31
HumanEval	单步 (<code>single_step</code>)	97.56 ± 0.00	1.000 ± 0.000	$64,938 \pm 116$	—
HumanEval	单步 + 修补 (<code>single_step_repair</code>)	99.19 ± 0.35	1.020 ± 0.003	$66,925 \pm 1,588$	8.19
HumanEval	分流 + 重链与救援 (<code>difficulty_aware_plus, v2, \tau=1.6</code>)	99.39 ± 0.00	1.067 ± 0.010	$75,151 \pm 1,064$	0.25
HumanEval	级联 (<code>workflow_cascade</code> , 默认基线)	99.80 ± 0.35	1.055 ± 0.010	$73,847 \pm 1,422$	—

注: MBPP 与 HumanEval 各档均为三 seed (42/78/132) 均值 $\pm$ 样本标准差。边际 ρ 为相对上一档, $\rho = \Delta p_{(\%)} / (\Delta t_{\text{tok}} / 10^4)$, 单位: 百分点/万 token; 当相邻两档总 token 不增 ($\Delta t \leq 0$) 时边际 ρ 无定义, 表中记为 “—”。表 11 的 $\rho_b^{(\text{cum})}$ 为相对单步档累计, 分母恒为单步至该档 token 增量, 可与边际 ρ 同时阅读。

沿关系。另在机制层面，E2 反馈形态对照已补充五 seed，见表 7，用于独立检验失败回注形式本身是否改变结论。

4.2.1 多模型对照

表 4 与表 5 进一步报告多组模型的分档通过率，用于检验同一协议下档位形态是否随基础模型发生系统性漂移。

总体而言，多数模型仍呈现「单步 → 单步 + 修补 → 分流 + 重链与救援 → 级联」逐级抬升的粗形态，但绝对分数与性价比因模型能力差异较大：强模型在中档即可获得较大部分收益，较弱模型则更依赖高档候选扩展。跨表比较时，建议以「是否越过同一类失败瓶颈」作为对齐轴，而不是仅比较不同模型在同档的终值分数；同时注意上表为单次 seed 的形态扫描，不宜与主表三 seed 区间做“谁高谁低”的硬比较。

4.3 机制验证

本节同时报告控制基线与模块消融：阈值与流程级消融仍以 MBPP 427 题为主（易饱和 HumanEval 上细粒度开关更易表现为个位数波动）；multiset 随机路由对照则在 HumanEval 164 与 MBPP 427 上各做一次配对运行（claude-sonnet-4-6，分流 + 重链与救援档 $b=0.6$ ，seed=42），以便与 LR 离线证据并列阅读。除表 7 与表 6 的说明外，其余 MBPP 消融表多为 seed=42 单次；与主表三 seed 结论并列时应区分“有区间统计的档位趋势”与“单次机制解剖”。

4.3.1 学习式路由控制基线

为回应“结构难度分能否替代规则分流、或能否直接预测运行失败”的问题，本文在与打分表对齐的 task 级样本上给出逻辑回归 5-fold 交叉验证对照（特征含五维子分与体量项等）。结果显示：direct_fail 与 need_reroute 的 ROC-AUC 分别约为 0.471 与 0.473，接近随机分类器；加入扩展静态特征后 AUC 约升至 0.51，增量有限。

正确解读。上述结果只能说明：在当前标签口径与任务规模下，用静态表特征去拟合“是否首轮失败/是否需要改道”这类运行标签的判别能力弱。它既不蕴含“规则分流优于任意随机路由”，也不蕴含

“规则分流优于任意学习式路由器”。其中，“规则 vs 随机”需要协议内、可复现的在线对照；本文在表 6 给出一种明确对照：在固定 easy/hard multiset（与同批 structural 相同的 hard 覆盖率）下随机置换题—槽位对应，再与 structural 同跑整卷。该对照仍不覆盖其它随机化形式（例如 Bernoulli(p) 独立抽样导致 hard 计数方差等），故讨论中避免将结论外推为“一切随机分流均劣于结构分流”。本文将规则分流定位为可解释、可阈值化的预算分配先验：其有效性由端到端通过率与 token 前沿及 τ 消融共同支持，边界由 LR 证据说明“不要把同一组静态分误读为失败预测器”，并由表 6 说明“在固定覆盖率 multiset 下，结构对齐可带来可观差异（且常伴随更高 token）”。

4.3.2 Multiset 随机路由整卷对照

表 6 报告 difficulty_aware_plus 档在 $b=0.6$ 、 $\tau=1.6$ 下的配对结果：structural 为默认结构分流；random_permute_matched 为第 3.3 节所述 multiset 置换对照。两列在各自数据集上 easy_tasks/hard_tasks 完全一致（MBPP: 180 easy / 247 hard; HumanEval: 12 easy / 152 hard），因而比较的是同一 hard 预算槽位 multiset 在不同题—槽位对齐方式下的端到端差异。

在本次单次配对下，MBPP 上 structural 较 multiset 随机置换多对 4 题，但总 token 与总调用更高，呈现典型的精度-成本交换；HumanEval 上通过率同为 163/164，随机置换略省 token（差异量级较小，宜结合易饱和和区间解读为“对齐效应不显著或噪声主导”）。因此表 6 应作为对 LR 证据的补充：LR 说明静态分不是好的失败分类器；本对照则说明在固定 hard 覆盖率 multiset 的前提下，结构—槽位对齐在更难、更杂的分布上仍可带来非零通过率差，但通常不是“免费午餐”。

4.3.3 反馈回注形态对照

为给出失败反馈压缩的独立因果证据，本文在相同模型、预算、任务规模与路由配置下，仅替换 repair 阶段的错误回注形态：compressed，默认压缩反馈，与 raw_traceback，完整 traceback。对照采用五 seed，42、78、132、256、512，并保持任务打乱开启，以减少单次偶然样本影响。结果见表 7。

表 4: 不同模型在 MBPP 上的预算分档结果（通过率%；列为第3.1节四档语义；seed=42 单次，与主表三 seed 均值非同一统计量）

模型	单步	单步 + 修补	分流 + 重链与救援	级联
claude-sonnet-4-6	82.51	86.65	88.52	91.72
gpt-5.4	72.13	77.75	79.39	85.48
gpt-5.3-codex	71.90	78.92	79.39	83.84
qwen3-coder-plus	71.90	76.58	79.63	81.97
qwen3-30b-a3b-instruct-2507	68.38	75.18	77.99	80.80
gemini-3.1-flash-lite-preview	73.54	76.81	79.39	80.09
qwen-plus	70.73	75.18	77.28	79.63

表 5: 不同模型在 HumanEval 上的预算分档结果（通过率%；列为第3.1节四档语义；seed=42 单次）

模型	单步	单步 + 修补	分流 + 重链与救援	级联
claude-sonnet-4-6	97.56	99.19	99.39	99.80
gemini-3.1-flash-lite-preview	97.56	98.78	99.39	98.78
gpt-5.3-codex	95.12	97.56	98.78	99.39
gpt-5.4	95.12	97.56	97.56	98.78
qwen3-coder-plus	95.73	97.56	98.78	98.17
qwen3-30b-a3b-instruct-2507	92.68	95.73	97.56	96.34
qwen-plus	92.68	97.50	96.34	95.12

表 6: Multiset 随机路由整卷对照（difficulty_aware_plus, $b=0.6$, claude-sonnet-4-6, seed=42 单次。random 列为 random_permute_matched。

数据集	structural（通过/总）	random multiset（通过/总）	structural 总 token	random 总 token
MBPP 427	383/427（89.70%）	379/427（88.76%）	267,878	234,912
HumanEval 164	163/164（99.39%）	163/164（99.39%）	77,683	76,563

注：同表汇总平均调用次数：MBPP 为 structural 677 次 vs random 640 次；HumanEval 为 178 vs 177 次。跑分落盘文件名含时间戳 20260512，命令行含 --difficulty_assignment structural 或 random_permute_matched。

表 7: E2 反馈形态对照（MBPP 427, difficulty_aware_plus, $b = 0.6$ ；五 seed，均值 $\pm$ 样本标准差）

回注形态	passed	pass@1(%)	总 token	结论
compressed	379.4 ± 3.85	88.852 ± 0.900	$229,713 \pm 7,558$	均值 token 更低；与对照组通过率差小于各自标准差量级
raw_traceback	379.0 ± 1.41	88.756 ± 0.334	$233,180 \pm 2,781$	均值 token 更高；未做 formal 显著性检验

五 seed 平均上, `compressed` 相对 `raw_traceback` 应按随机波动区间解读, 不宜单独上升为稳定优势的通过率差异仅为 +0.096 个百分点 (约 0-1 题量级), 且两组 `passed` 与 `pass@1` 的样本标准差带相互重叠; 总 token 均值差约 3,467, 同样未报告 formal 显著性检验。因此本文不主张 “`compressed` 在统计意义上严格更准”, 而将其表述为: 在观测到的均值意义上两者精度几乎无差, `compressed` 在成本侧更稳; 部署上可把 `compressed` 作为上下文更短的回注默认, 并保留按任务分布做小规模回归切换的权利。

4.3.4 分流 + 重链与救援档: 难度阈值 τ

表 8 在固定其余流程的前提下扫描 τ , 观察 `easy/hard` 占比、`passed` 与 token 的联动关系。

从表 8 可见, τ 实质上控制 `hard` 覆盖率, 从而在精度与成本之间形成连续可调的前沿: 较低阈值提高上限但更耗 token; 较高阈值压缩成本但可能损伤精度。默认点 $\tau = 1.6$ 位于折中区间, 后文流程消融均在该点展开。

4.3.5 分流 + 重链与救援档: 流程开关

表 9 在 $\tau = 1.6$ 的默认点处, 对 `spec`、`cot` 以及两级救援 (候选救援与 `refine` 救援) 等进行开关式对比, 用以识别主贡献模块与边际模块。

表 9 的主次结构相对清晰: `hard_spec` 对主链下限影响最大; `hard_cot` 与候选救援提供次级增益, 其中候选救援同时带来较明显的 token 上升; `refine` 救援的边际收益相对有限。该排序与「先把搜索空间收窄, 再考虑扩展推理与候选」的直觉一致。

4.3.6 级联档: 组织方式与扩展强度

表 10 聚焦级联档内部的关键开关。本文主配置为固定默认级联基线, 因此消融只比较在该基线上对 `extra` 与 `refine` 相关模块的开关, 用以解释高预算下性能上界主要由哪些环节支撑。

表 10 表明, `extra` 所对应的候选扩展几乎是高预算性能的必要条件; 一旦移除, 系统迅速退化为接近单轮调用的成本结构, 但通过率亦随之断崖式下降。另一方面, 在默认级联基线上, `refine` 相关开关体现的是精度-成本微调而非主增益来源: 关闭一次 `refine` 可显著节省 token, 但会带来轻微降分; 语义门控版本则提供折中点。需要注意, 诸如 “390/427 vs 392/427” 这类约 2 题 (同量级地, 个位数题差)

结论。该观察与「高预算的关键在候选扩展, 次要收益来自后处理组织」一致。更一般地, 这一负面结果说明 “阶段更多” 并不自动带来收益: 当主链路已经由候选扩展覆盖主要增益后, 附加后处理步骤可能只提供边际修饰, 甚至在预算受限时挤占关键调用配额。

4.3.7 证据边界与主结论的对应关系

本文关注的是 API 成本约束下的可落地编排协议: 在同等资源约束下如何组织已有能力, 使通过

关于证据权重: 分档有效性、跨模型形态与双基准趋势, 应以表 3 的三 seed 区间为主证据; 多模型分档表用于单次 seed 下的形态扫描, 不宜与主表做硬数值对齐。模块主次与阈值-成本前沿, 以 MBPP 上的表 8-10 为机制材料 (多为单次 seed)。Multiset 随机路由对照 (表 6) 为双基准、单次 seed 的配对整卷结果, 用于补充 LR 叙述而非替代主表区间统计。

4.4 成本-收益前沿分析

综合主结果与消融, 可将本文系统在代码生成中的行为概括为三条可复述的观察。(1) 双基准主结果表明, 从单步到级联带来的主要增量在 MBPP 上更显著, 在 HumanEval 上更多体现为残差段的稳健收尾。(2) 分流 + 重链与救援档的阈值与流程消融表明, 预算首先应花在能把搜索空间收窄的模块上 (如 `spec` 主链), 其次才是可选的推理加深与候选救援。(3) 级联档消融表明, 高预算的核心不是堆叠更多阶段名称, 而是候选扩展 (`extra`); 在默认级联基线上, `refine` 相关模块主要承担精度与 token 的细粒度折中。

从工程部署角度看, 可形成两级策略: 在分流 + 重链与救援档优先通过阈值调节获得成本可控的稳定增益 (推荐保留 `hard_spec`, 并按预算决定 `cot` 与救援层深度); 在级联档将 `extra` 作为不可删除模块, 并在默认级联基线下按需调节 `refine` 强度。相较「无差别增加调用次数」, 这种中预算做分流与修补、高预算做候选扩展的结构更接近可落地的 API 部署逻辑。部署前仍建议在 HumanEval 与 MBPP 两类分布上做小规模回归, 以避免只优化其中一种失

表 8: 分流 + 重链与救援档阈值消融 (MBPP 427; v2 打分器, 第三档其余配置固定; seed=42 单次)

阈值 τ	easy/hard	pass@1	平均调用	总 token	结论
$\tau=1.0$	92 / 335	382/427	1.646	255,523	精度最高, 但成本明显更高
$\tau=1.1$	127 / 300	381/427	1.597	243,529	轻微降分, 成本下降
$\tau=1.4$	187 / 240	380/427	1.511	220,825	进入更平衡区间
$\tau=1.6$	213 / 214	381/427	1.489	217,510	默认点, 精度/成本折中
$\tau=1.7$	224 / 203	379/427	1.476	217,534	再提高阈值出现降分
$\tau=1.8$	254 / 173	376/427	1.447	205,909	成本最低, 但精度损失明显

表 9: 分流 + 重链与救援档流程消融 (MBPP 427; 阈值固定 $\tau=1.6$, v2 打分器; seed=42 单次)

配置	pass@1	平均调用	总 token	结论
完整流程 (all-on)	381/427	1.489	217,510	作为分流 + 重链与救援档基线
-hard_spec	360/427	1.164	148,549	spec 为主贡献模块, 关闭后精度显著下降
-hard_cot	375/427	1.436	202,847	cot 提供中等增益, 成本可控
-plus_candidate_rescue	377/427	1.283	167,780	候选救援有增益, 且是主要额外成本来源
-plus_refine_rescue	378/427	1.450	207,200	refine 层贡献较小、边际收益有限

表 10: 级联档流程消融 (cascade, MBPP 427; seed=42 单次)

配置	pass@1	平均调用	总 token	结论
主配置 (默认级联基线)	392/427	1.595	299,935	级联档默认基线
-extra	355/427	1.000	108,610	精度大幅下降, extra 为核心增益模块
-refine_once	387/427	1.529	280,307	轻微降分, 显著省 token
+semantic_refine_gate	390/427	1.541	280,784	折中方案: 接近最优精度且成本更低

败形态。

最后，引入一个仅用于事后解释、而不用于训练或路由的标量。设某策略相对其前一档带来的通过率增量为 Δp ，额外 token 开销为 Δt ，定义经验性收益率

$$\rho = \frac{\Delta p}{\Delta t}. \quad (6)$$

为便于横向比较，本文进一步报告“相对单步档”的累计收益率

$$\rho_b^{(\text{cum})} = \frac{p_b - p_{\text{single}}}{t_b - t_{\text{single}}}, \quad (7)$$

并以“每万 token 带来的通过率提升百分点”呈现。表 11 给出主结果中的对应数值。

当 ρ 较高时，通常意味着新增 token 换来了更显著的正确率提升；当 ρ 明显下降时，则提示系统可能进入「继续加预算但收益变缓」的区间。结合表 3，HumanEval 更容易出现 Δp 较小但仍具工程意义的残差段；MBPP 则更常出现 Δp 与 Δt 同步变化、需要比较前沿的情形。由此，级联档更自然地扮演高预算探索上界，而分流 + 重链与救援档更自然地扮演中预算折中方案；二者在不同基准上的可感知幅度可能不同，但角色分工一致。

5 讨论与结论

本文的主结论是：在 API 成本约束下，代码生成系统的收益来自“把预算投向不同失败形态”，而不是统一增加调用次数。双基准主结果表明，同一协议下从单步到级联形成稳定前沿；中档主要提升来自分流与失败后修补，高档主要提升来自候选扩展与筛选组织。

机制证据与主结论需要分层阅读。LR+CV 在静态特征上接近随机，只能说明离线失败标签难以被表特征稳定预测；表 6 给出固定 multiset 的整卷随机置换对照，用于回答「在 hard 覆盖率不变时，结构一槽位对齐是否仍有边际」：MBPP 上 structural 较 random 通过率更高但 token 亦更高，HumanEval 上通过率相同而 token 差较小。该对照不等价于所有可能随机路由；结论应限定在本文实现与单次 seed 观测内。反馈形态多种子对照（表 7）显示两组均值通过率差异落在各自标准差量级内，未做 formal 检验；在此证据强度下，更稳妥的部署表述是“compressed 回注在均值上未显示精度损失，同时平均 token 更低”，而非“默认工程形态必然更优”。

对部署实践而言，可形成清晰建议：中预算优先保留 hard_spec 与失败回注修补，高预算将新增计算优先分配给 extra 候选扩展，再按需开启 refine 微调。这一策略与第 4.4 节 ρ 前沿一致，即在可接受 token 增量下换取更高通过率。

当前证据边界也明确：细粒度消融仍以 MBPP 为主；尽管主表中的 MBPP 与 HumanEval 均已补齐四档三 seed 区间，multiset 随机路由对照仍为单次 seed=42，若需与主表同级的区间统计可扩展多 seed 重复配对。跨 API 版本漂移仍待系统覆盖。后续工作将优先扩展跨 API 版本稳定性验证，并在同一函数级代码生成口径下做更长周期回归。

A 难度子项与权重依据的补充统计

为回应“子项权重是否只有直觉依据”的问题，表 12 汇总了本文已使用结果文件中的代表性统计（不新增训练流程）：一类是子项与结构分档/运行标签的 Spearman 相关，另一类是学习式路由基线在不同特征块上的交叉验证 AUC。该表用于说明“哪些信号更像结构先验、哪些信号对运行失败预测有限”，而非把静态分数解释为强预测器。

该补充表支持本文的方法定位：难度子项用于可解释的预算分配先验；LR 证据用于说明不要把同一组静态分误读为运行失败分类器，并不推出规则分流相对随机分流的优劣。主结论仍以端到端通过率、调用与 token 前沿为准。

参考文献

- [1] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code[EB/OL]. arXiv:2107.03374, 2021.
- [2] AUSTIN J, ODENA A, NYE M, et al. Program synthesis with large language models[EB/OL]. arXiv:2108.07732, 2021. DOI:10.48550/arXiv.2108.07732.
- [3] LI Y, CHOI D, CHUNG J, et al. Competition-level code generation with AlphaCode[J]. Science, 2022, 378(6624): 1092-1097. DOI:10.1126/science.abq1158.

表 11: 预算分档的经验收益率前沿（相对单步档累计；单位：每万 token 提升百分点；数值与表 3 三 seed 均值对齐）

数据集	档位	Δp (百分点)	Δt (token)	$\rho_b^{(\text{cum})} \times 10^4$
MBPP	单步 + 修补 (single_step_repair)	4.14	27,284	1.52
MBPP	分流 + 重链与救援 (difficulty_aware_plus)	6.01	104,405	0.58
MBPP	级联 (workflow_cascade)	9.21	207,255	0.44
HumanEval	单步 + 修补	1.63	1,986	8.19
HumanEval	分流 + 重链与救援	1.83	10,213	1.79
HumanEval	级联	2.24	8,909	2.51

表 12: 难度子项与权重依据的补充统计（MBPP；数据源为离线路由证据与特征实验报告）

统计项	指标	数值	n	解读
$\rho(\text{structural_hard}, \text{test_score})$	Spearman	0.5696	427	与结构分档中等正相关，支持测试复杂度在结构先验中的核心地位
$\rho(\text{direct_fail}, \text{test_score})$	Spearman	-0.0337	383	与首轮失败相关性接近 0，说明静态子分难以直接预测运行失败
$\rho(\text{direct_fail}, \text{boundary_score})$	Spearman	0.0483	383	方向与量级均弱，不能单独作为失败预测证据
$\rho(\text{direct_fail}, \text{extent_score})$	Spearman	0.0669	383	体量项与首轮失败的单变量关联有限
need_reroute (v2 五子分)	LR 5-fold AUC	0.450 ± 0.103	383	仅用子分时接近随机
need_reroute (扩展静态)	LR 5-fold AUC	0.520 ± 0.108	383	加入扩展静态后略有提升，但总体仍接近弱信号区间

- [4] CHEN X, LIN M, SCHARLI N, et al. Teaching large language models to self-debug[C/OL]//The Twelfth International Conference on Learning Representations. 2024.
- [5] SHINN N, CASSANO F, BERMAN E, et al. Reflexion: language agents with verbal reinforcement learning[C/OL]//Advances in Neural Information Processing Systems 36. 2023.
- [6] GOU Z, SHAO Z, GONG Y, et al. CRITIC: large language models can self-correct with tool-interactive critiquing[C/OL]//The Twelfth International Conference on Learning Representations. 2024.
- [7] LI D, CAO S, CAO C, et al. S*: test time scaling for code generation[C]//Findings of the Association for Computational Linguistics: EMNLP 2025. Suzhou: Association for Computational Linguistics, 2025: 15964-15978. DOI:10.18653/v1/2025.findings-emnlp.865.
- [8] HENDRYCKS D, BASART S, KADAVATH S, et al. Measuring coding challenge competence with APPS[C]//Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks. 2021.
- [9] LIU J, XIA C S, WANG Y, et al. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation[C]//Advances in Neural Information Processing Systems 36. 2023.
- [10] JAIN N, HAN K, GU A, et al. LiveCodeBench: holistic and contamination free evaluation of large language models for code[EB/OL]. arXiv:2403.07974, 2024.
- [11] LAI Y, LI C, WANG Y, et al. DS-1000: a natural and reliable benchmark for data science code generation[C]//Proceedings of the 40th International Conference on Machine Learning. 2023: 18319-18345.
- [12] HUANG D, ZHANG J M, LUCK M, et al. AgentCoder: multi-agent-based code generation with iterative testing and optimisation[EB/OL]. arXiv:2312.13010, 2024. DOI:10.48550/arXiv.2312.13010.
- [13] MADAAN A, TANDON N, GUPTA P, et al. Self-Refine: iterative refinement with self-feedback[C/OL]//Advances in Neural Information Processing Systems 36. 2023.
- [14] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Advances in Neural Information Processing Systems 35. 2022.
- [15] WANG X, WEI J, SCHUURMANS D, et al. Self-consistency improves chain of thought reasoning in language models[C/OL]//The Eleventh International Conference on Learning Representations. 2023.
- [16] YAO S, ZHAO J, YU D, et al. ReAct: synergizing reasoning and acting in language models[C/OL]//The Eleventh International Conference on Learning Representations. 2023.
- [17] YAO S, YU D, ZHAO J, et al. Tree of thoughts: deliberate problem solving with large language models[C]//Advances in Neural Information Processing Systems 36. 2023.
- [18] SCHICK T, DWIVEDI-YU J, DESSI R, et al. Toolformer: language models can teach themselves to use tools[C]//Advances in Neural Information Processing Systems 36. 2023.
- [19] LU S, GUO D, REN S, et al. CodeXGLUE: a machine learning benchmark dataset for code understanding and generation[C]//Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks. 2021.
- [20] CHEN B, ZHANG F, NGUYEN A, et al. CodeT: code generation with generated tests[C/OL]//The Eleventh International Conference on Learning Representations. 2023.
- [21] LE H, WANG Y, GOTMARE A D, et al. CodeRL: mastering code generation through pre-

trained models and deep reinforcement learning[C]//Advances in Neural Information Processing Systems 35. 2022.

- [22] ADNAN M, XU Z, KUHN C C N. Large language model guided self-debugging code generation[EB/OL]. arXiv:2502.02928, 2025.
- [23] CHEN X, TAO Z, ZHANG K, et al. Revisit self-debugging with self-generated tests for code generation[C]//Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2025: 18003-18023. DOI:10.18653/v1/2025.acl-long.881.